

MIS5303 Securing Software

Lab 4 — Dangerous Functions, Insecure File Handling & Threat Modelling

Assessment 1 — Supervised Practical Lab Submission

Field	Detail
Name	Ashraful Islam
Student ID	240135
Lab number	Lab 4 (Week 5)
Date of completion	[dd/mm/2026]
Subject	MIS5303 Securing Software
Lecturer	Dr Omid Haass

Note on scope: this lab targets the two dangerous features - the /admin/ping command execution and the /upload file handling. To match the lab sheet's directory-traversal test I added a /uploads/<filename> route so uploaded files can be served back. Other flaws are out of scope for this lab.

1. Findings — Dangerous Features

ID	Vulnerability	Where	OWASP / CWE	Severity	Why it is risky
F1	OS command injection	/admin/ping	A03 / CWE-78	Critical	Input concatenated into a shell string with shell=True, so extra commands run with the server's privileges
F2	Unrestricted file type upload	/upload	A05 / CWE-434	High	Any file type (.py, .exe, .php) is accepted, enabling malware upload or a web shell
F3	Path traversal on upload	/upload	A01 / CWE-22	High	A ../ filename escapes the uploads folder and can overwrite files elsewhere on the server

F1 — Command injection (before)

Step: in the Ping a Host box I entered 127.0.0.1 & dir. Windows treats & as a command separator, so the app ran ping AND the dir command, listing the server's files. An attacker could run any command the server can.



Figure 1: command injection - '127.0.0.1 & dir' runs ping and then dir, exposing a directory listing.

F2 — Unrestricted upload (before)

Step: I uploaded hack.py, a Python script. The app accepted it with no extension or type check. A dangerous executable file type should never be accepted.



Figure 2: the app accepts a .py script upload with no restriction.

F3 — Path traversal (before)

Step: using a scripted request I uploaded a file named `../../hacked.txt`. The traversal escaped the uploads folder and wrote `hacked.txt` into the parent S2B1 directory, shown below in File Explorer.

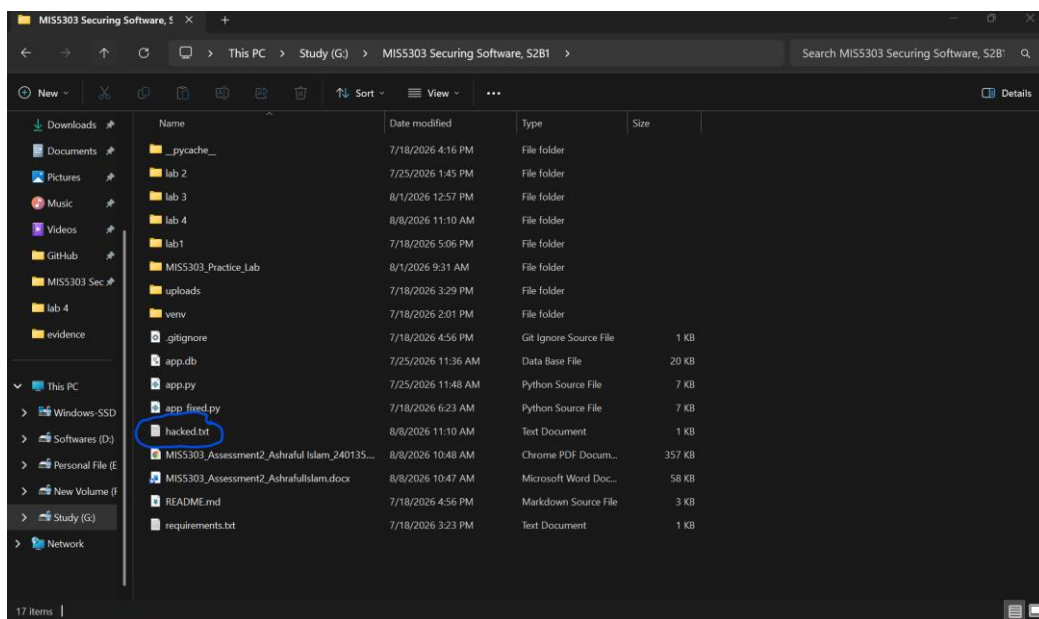


Figure 3: the uploaded '`../../hacked.txt`' escaped the uploads folder and landed in the parent directory (circled).

2. Remediation — Secure Code (Before / After)

2.1 Command injection → validate + argument list

Before:

```
result = subprocess.run("ping -n 1 " + host, shell=True, ...)
```

After:

```
if not re.match(r"^[A-Za-z0-9.\-]{1,253}$", host):
    return "Invalid host..." # reject ; & | and spaces
subprocess.run([ping_bin, "-n", "1", host], shell=False, timeout=5)
```

Why: input is validated against a strict host/IP pattern, and the command runs as an argument LIST with shell=False, so shell metacharacters like & are never interpreted.

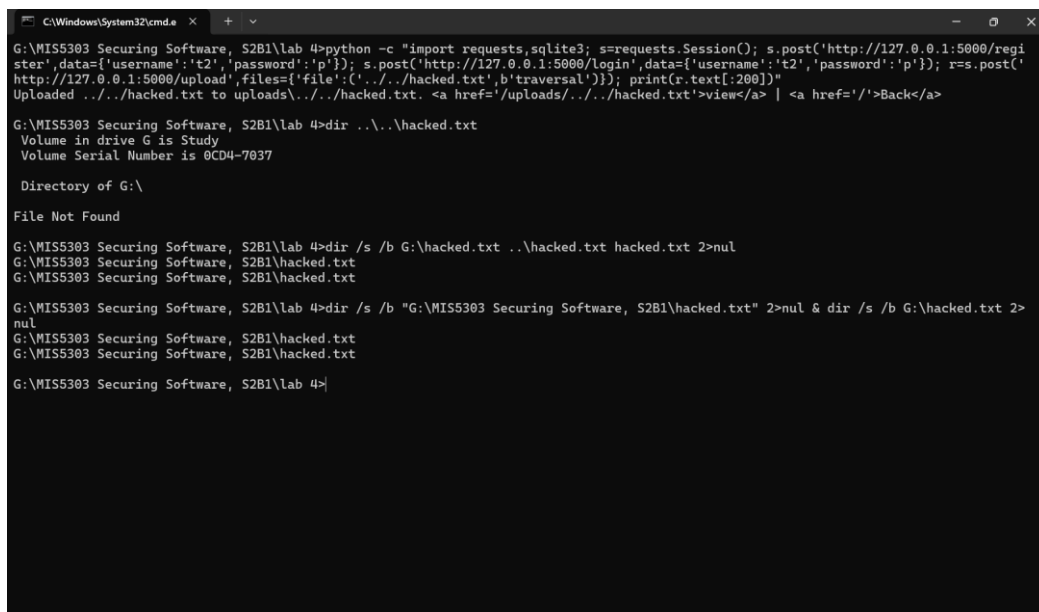
2.2 File upload → secure_filename + extension allow-list

```
filename = secure_filename(f.filename or "") # strips ../ and paths
ext = os.path.splitext(filename)[1].lower()
if not filename or ext not in ALLOWED_EXTENSIONS: return "...", 400
# ALLOWED_EXTENSIONS = {'.txt', '.jpg', '.jpeg', '.png', '.pdf'}; MAX_CONTENT_LENGTH = 2 MB
```

Why: secure_filename removes any path components (defeating ../ traversal), the allow-list blocks executable types, and a size cap limits abuse. Files are served with send_from_directory, which also rejects traversal.

Re-test — after remediation

Attack repeated	Before	After (fixed)
Ping '127.0.0.1 & dir'	runs dir (injection)	'Invalid host' (rejected)
Upload hack.py	accepted	'File type not allowed' (400)
Upload evil.txt	accepted	still accepted (safe type)
Upload ../../hacked.txt	escapes to parent folder	stays inside uploads/ (contained)



```
C:\Windows\System32\cmd.exe
G:\MIS5303 Securing Software, S2B1\lab 4>python -c "import requests,sqlite3; s=requests.Session(); s.post('http://127.0.0.1:5000/register',data={'username':'t2','password':'p'}); s.post('http://127.0.0.1:5000/login',data={'username':'t2','password':'p'}); r=s.post('http://127.0.0.1:5000/upload',files={'file':( '../../hacked.txt',b'traversal')}); print(r.text[:200])"
Uploaded ../../hacked.txt to uploads\../../hacked.txt. <a href='/uploads/../../hacked.txt'>view</a> | <a href='/'>Back</a>

G:\MIS5303 Securing Software, S2B1\lab 4>dir ../../hacked.txt
Volume in drive G is Study
Volume Serial Number is 0CD4-7037

Directory of G:\

File Not Found

G:\MIS5303 Securing Software, S2B1\lab 4>dir /s /b G:\hacked.txt ../../hacked.txt hacked.txt 2>nul
G:\MIS5303 Securing Software, S2B1\hacked.txt
G:\MIS5303 Securing Software, S2B1\hacked.txt

G:\MIS5303 Securing Software, S2B1\lab 4>dir /s /b "G:\MIS5303 Securing Software, S2B1\hacked.txt" 2>nul & dir /s /b G:\hacked.txt 2>nul
G:\MIS5303 Securing Software, S2B1\hacked.txt
G:\MIS5303 Securing Software, S2B1\hacked.txt

G:\MIS5303 Securing Software, S2B1\lab 4>
```

Figure 4: after the fix - '127.0.0.1 & dir' is rejected with 'Invalid host'.

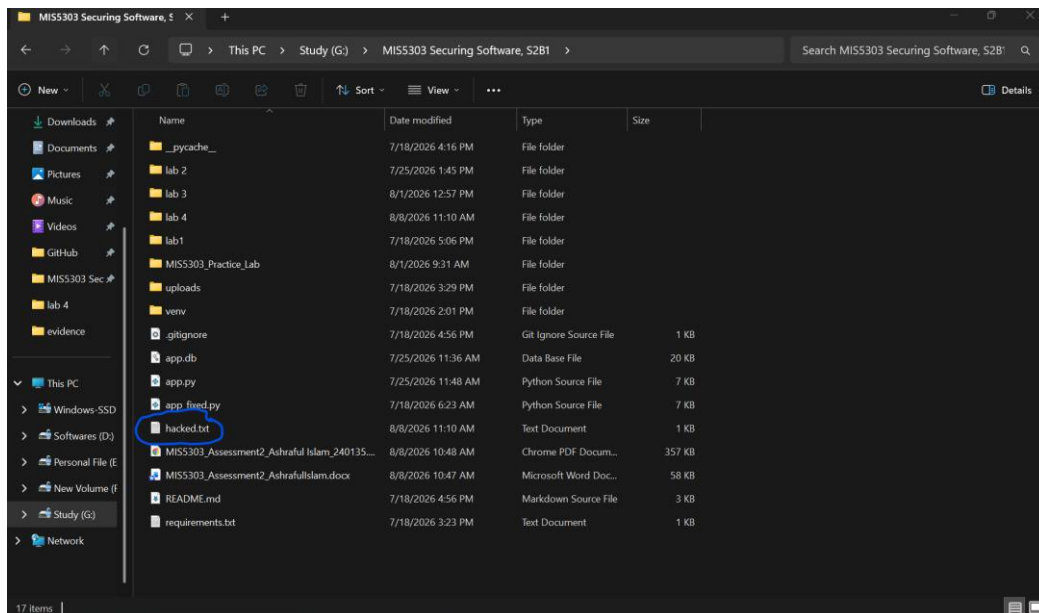


Figure 5: after the fix - the .py upload is blocked ('File type not allowed').



Figure 6: after the fix - a safe .txt file still uploads successfully ('Uploaded evil.txt safely').

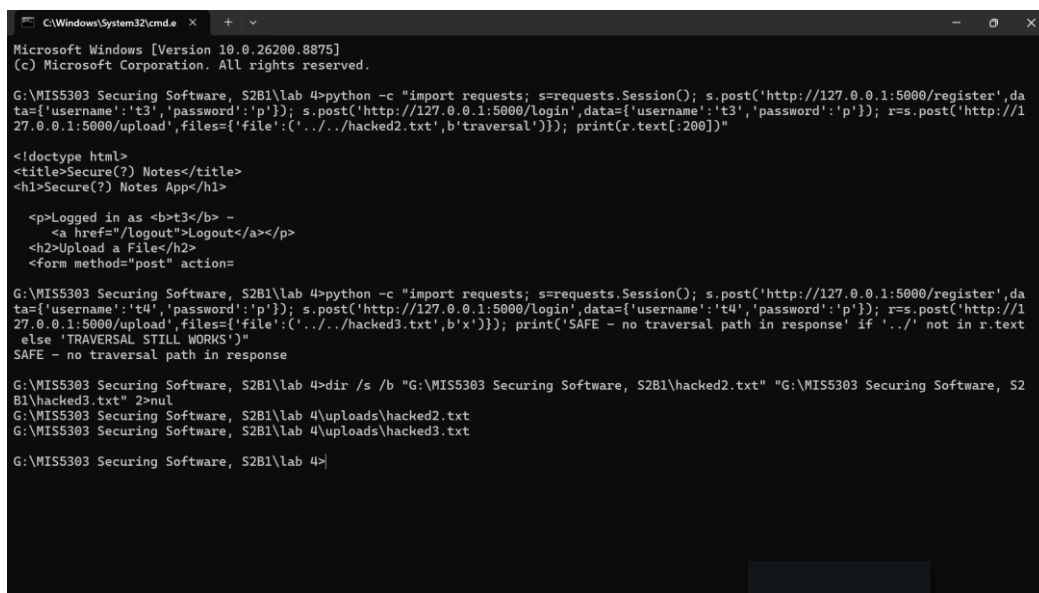


Figure 7: after the fix - the ../.. upload is contained; both files stay inside lab4\uploads\ instead of escaping.

3. Threat Model & Design Proposal

STRIDE threat model for the two dangerous features:

Feature	STRIDE	Threat / attack vector	Risk / impact	Mitigation
/admin/ping	Tampering / EoP	Command injection via ; & in input	Arbitrary OS command execution as the server	Input validation; subprocess arg list, shell=False
/upload	Tampering	Path traversal via ../filename	Overwrite or place files outside uploads/	secure_filename; send_from_directory
/upload	Elevation of Privilege	Upload .py/.exe/.php (web shell / malware)	Remote code execution; client infection	Extension allow-list; MIME check; size limit
/upload, /ping	Repudiation	No logging of misuse	Attacks go undetected	Log and alert on rejected input/uploads

Design proposal

The most important architectural change is to treat both features as untrusted-input boundaries and never pass user input to a shell or a filesystem path unchecked. For command execution, avoid shells entirely - use argument lists, or better, a native library (e.g. a ping library) so no shell is involved. For uploads, validate on an allow-list, rename files to server-generated names, store them outside the web root, and scan them. In production I would sandbox the ping feature (container or restricted service account), add a Web Application Firewall, cap upload size and rate, and log every rejected request so misuse is detected. Frameworks/libraries: werkzeug secure_filename, Flask-WTF for CSRF, and a malware-scanning step for uploads.

4. Individual Reflection

In this lab I identified and addressed two potentially dangerous aspects of the application. The first was command injection in the ping tool. Entering 127.0.0.1 & dir into the Host box caused the app to ping and also list the files on the server via the dir command. This is dangerous because, running as the server, an attacker could run whatever command he wishes — reading, deleting or stealing data. I fixed it on two points: first I sanitised the input so it could only contain letters, digits, dots and hyphens (no & or ;), and second I ran the command as an argument list with shell=False, so shell characters are never interpreted. After the fix, 127.0.0.1 & dir is rejected with "Invalid host."

The second was insecure file upload. The app accepted a .py file, and could be exploited via the ../ traversal trick to write into the parent directory. I resolved that by using secure_filename, which removes the ../, and an allow-list that only permits safe types such as .txt and .jpg. After this patch, the .py is not allowed, a .txt uploads as normal, and the ../.. file stays inside the uploads folder.

The hardest to fix was the command injection, since it executes actual system commands; the file upload felt less severe by comparison. The single most important lesson is that both bugs came from a common error — mistaking a user's input for something safe to pass directly to a shell or a file path. You should never do that; always validate and check the input first. I also learned to re-test each fix by repeating the same attack to confirm it was blocked, and all three were

5. Peer Review

Reviewer: Z N Zakaria (Student ID 240134). Peer feedback on this Lab 4 submission.

Did the fixes close the vulnerabilities?

Yes — all three re-tests demonstrate the "fixes work" The command injection is rejected ("Invalid host"), the upload of .py is rejected, (but .txt works) and the ../../ stays in the uploads directory. An argument list with the shell=False will prevent the injection problem. One idea: validate on server the host is "real/feasible" reachable format, not just the char pattern

Is the threat model complete and realistic?

The STRIDE table is pretty complete in term of covering main threats: injection, traversal and upload of malware. It would have been nice to see a log misuse row added in as Repudiation. It might be improved by introducing another DoS entry for the upload (large or many files) because there is a size limit but it is not formed as a threat.

Polishing / presentation

Screenshots are in good, clear, and understandable format. Suggest: Add the figure number right on the images themselves and, early in the presentation, a comment that the scope will not be apparent to the marker, but that the app was extended with an /uploads/ route for the traversal test, a requirement in lab sheet

Reviewer signature: Z N Zakaria Date: 08-Aug-2026

6. Video Presentation Script (≈5 minutes)

Maps 1:1 to this report so the video and document stay coherent. Demonstrate live and narrate in your own words.

[0:00–0:30] Intro

- Name + student ID; state this is MIS5303 Lab 4 - dangerous functions, insecure file handling and threat modelling.

[0:30–2:15] Vulnerabilities (before)

- F1: in the Ping box enter 127.0.0.1 & dir; show ping AND a file listing appear (Figure 1). Say why it's critical.
- F2: upload hack.py; show it's accepted (Figure 2).
- F3: show the ../../hacked.txt upload escaped into the parent folder in File Explorer (Figure 3).

[2:15–4:00] Fixes + re-test (after)

- F1: show the validation + shell=False code; enter 127.0.0.1 & dir again -> 'Invalid host' (Figure 4).
- F2: upload hack.py -> 'File type not allowed' (Figure 5); upload evil.txt -> 'Uploaded safely' (Figure 6).
- F3: repeat the ../../ upload; show the file now stays inside uploads/ (Figure 7).

[4:00–5:00] Threat model + reflection

- Show the STRIDE table; name one threat and its mitigation.
- Close with one honest sentence: which risk was hardest and the key lesson.

7. Submission Checklist

- Cover details + date filled in
- F1–F3 documented with BEFORE screenshots (Figures 1–3) — DONE
- Secure code shown for both fixes — DONE
- Re-test table + AFTER screenshots (Figures 4–7) — DONE
- STRIDE threat model + design proposal — DONE
- Reflection written in my own words — TO DO
- Peer review by Z N Zakaria (Part 5) — TO DO
- Video recorded, coherent with this document — TO DO
- GenAI cover sheet completed and signed — TO DO
- app.py (vulnerable) and app_fixed.py both included — DONE